Natural Language Processing Based on a Text Graph Convolutional Network*

Vitor César Moreira Pereira[1][0000−0001−6132−5077] and
Leandro Nunes de Castro[1][0000−0003−3409−4589]

Natural Computing and Machine Learning Laboratory (LCoN), Mackenzie Presbyterian University, São Paulo, SP 01302-907, Brasil
vitormeriat@hotmail.com
lnunes1974@gmail.com

Abstract. Abstract. Deep Learning (DL) has been one of the preferred techniques for Natural Language Processing (NLP) applications. Due to its nature, a text can be better represented in a graph structure, when compared with the classical feature-based representations. Therefore, several researchers have explored the use of Graph Neural Networks (GNN) for text analysis. GNNs show excellent results in text classification tasks, given their property of capturing contextual and global information in a corpus. The Text Graph Convolutional Network (TGCN) showed the ability to outperform traditional NLP methods in benchmark classification tasks. However, this method has a very high memory cost for the text graph construction. By exploring the results of text representations, we propose a new method to generate a text graph, capable of influencing the result of the TGCN, leading to a reduced use of memory.

Keywords: Graph Neural Network · Deep Learning · NLP.

1 Introduction

In NLP, one of the most performed activities is text classification, being applied in several areas, such as SPAM detection, news categorization, topic analysis, question answering, among others [1]. Traditional text classification techniques involve textual representation and feature selection, followed by a classifier that uses these features to generate predictions. The text classifier has to assign labels to documents or text segments (e.g., sentences or paragraphs) [2].

The traditional approach based on manual feature extraction, like bag-of-words (BoW), introduces several limitations. An example is a need for analysis and domain knowledge for resource selection. As an alternative, neural approaches based on a model for learning embeddings to map the text into a low-dimensional vector have been explored [3]. The successful application of word embeddings techniques, combined with deep learning, allowed approaches based on Recurrent Neural Networks (RNN) [4] and Convolutional Neural Networks (CNN) [5] to achieve better results in text classification problems.

* Supported by FAPESP, CNPq and MackPesquisa

Recently, Graph Neural Networks (GNNs) have attracted a lot of attention, being applied in several areas, such as recommender systems, combinatorial optimization, drug discovery, and text classification. GNNs are designed to work directly with irregular graph data, having the ability to explore global resources and information [9]. Many domains are naturally worked on in graph structures, what is not the case with text data, which have sequential and contextual characteristics. Then, a construction phase of the text graph from the free text becomes necessary. Although several techniques can be applied for the construction of the text graph, these representations directly influence the model result, as will be discussed in this work.

Deep learning has been the dominant approach in performing many tasks in NLP [7]. The use of Word Embeddings techniques [3] brought new advances, leading to the state of the art for text classification. These techniques are based on the ability to model a text given its structural sequence characteristics. Thus, the ability to model texts in graphs allows the exploration of new features expressed by the graph topology [8]. However, there are challenges in relation to the use of graph networks applied to NLP, such as the richer exploration of graphs (for directed and heterogeneous graphs), and the high consumption in terms of memory, given that the modeling of text in graphs can generate a large number of edges that, when represented in matrix terms, can be computationally expensive.

2 Methods

2.1 Graph Convolutional Networks (GCN)

Graph Convolutional Networks (GCN) are a generalization of Convolutional Neural Networks (CNN) for non-Euclidean domain data and received several extensions over the past years [9], mainly focusing on capturing local correlations in graph signals. Its purpose is to operate directly on graph data.

Formally, for a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, $\mathcal{V}$ is a set of vertices and $\mathcal{E}$ is a set of edges. Each node i has a set of characteristics x_i that can be represented by a matrix $X \in \mathbb{R}^{N \times d}$, where N is the number of nodes and d the dimensionality of each node's feature vector. The connection relationship between the nodes forms an adjacency matrix A of dimension $N \times N$. $X \in \mathbb{R}^{N \times d}$ and $A \in \mathbb{R}^{N \times N}$ are the inputs of the GCN layer. The propagation of information between the GCN layers is given by Eq. 1.

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}H^{(l)}W^{(l)}\right) \tag{1}$$

where $\tilde{A}$ is the adjacency matrix, $\tilde{D}$ is the degree matrix, and $H^{(l)}$ represents the characteristics of the lth layer. The activation function (ReLU) is denoted by σ. H is the resource matrix and W is the trainable weights matrix.

2.2 Text Graph Convolutional Network (TGCN)

The TGCN proposed by [10] achieved cutting-edge performance in some datasets for text classification. A text graph must be constructed based on the sequential and contextual relationships of the words. Then, a graph convolutional network is used to learn embeddings of words and documents, extracting more contextual, semantic, and syntactic information [11]. The advantage of TGCN is the ability to exploit global resources. By working with graphs constructed from free text, TGCN proposes to learn embeddings of words and documents together, acting on the entire corpus modeled as a heterogeneous graph.

The construction of the input text graph for the TGCN considers the entire corpus, creating word nodes and document nodes. To create an edge between two-word nodes, co-occurrence information is used by measuring Pointwise Mutual Information (PMI), a popular measure for word associations. An edge is only created if PMI is positive (PMI > 0), where the edge receives a value as weight. The edge between a word node and a document node is constructed using TF-IDF, a value that is also assigned as the edge weight. For the construction of the adjacency matrix A, the diagonal elements are defined as 1, due to the inclusion of an auto-loop necessary to obtain the node information.

The resource matrix $X = I$ is an identity matrix that feeds into a two-layer GCN used to co-learn embeddings for words and documents. The second layer preserves the characteristics of the graph and feeds a softmax classifier:

$$Z = \text{softmax}(\tilde{A}\ \text{ReLU}\ (\tilde{A}XW_0)W_1) \tag{2}$$

where $E_1 = \tilde{A}XW_0$ and $E_2 = \tilde{A}\ \text{ReLU}\ (\tilde{A}XW_0)W_1$ represent the first and second layers of document and word embedding, respectively. The application of the cross-entropy error is defined as the loss function for all labeled documents. Fig. 1 illustrates the TGCN. The final text graph is given by $\mathcal{G} = (\mathcal{D}_{rep}, \mathcal{W}_{rep}, \mathcal{E})$, where $\mathcal{D}_{rep}$, is the document representation set, $\mathcal{W}_{rep}$ is the word representation set, $\mathcal{E}$ is the edge set of words and documents, and colors correspond to labels.

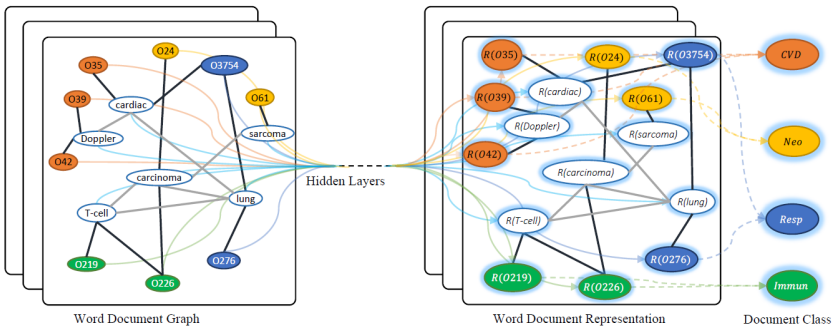


Fig. 1. The architecture of TGCN [10]

2.3 Building the Text Graph

The creation of static graphs in an attempt to model syntactic and semantic structures for texts has been widely explored in order to extract information to assist in NLP tasks [15]. This section describes the text graph construction, presenting the related word filter layer created to extract the properties that make the text more representative for the classification context, with a smaller graph, which reduces memory storage and training time.

The construction of the standard TGCN text graph is based on the information obtained from two measures: 1) Point-wise Mutual Information (PMI) [16], which is responsible for determining the edges between word nodes, and 2) Term Frequency-Inverse Document Frequency (TF-IDF) [17], which determines the edges between word nodes and document nodes. To build the text graph, TF-IDF was used to define the edges between words and documents. Other measures were tested, such as the TW-IDF [18], but the results generated by the TF-IDF were far superior. To create the edges between word nodes, we used the PMI value between two terms to determine if there is some level of correlation considering the entire corpus. The PMI value is given by:

$$\text{PMI}(i,j) = \log p(i,j) \;/\; p(i)\, p(j) \tag{3}$$

$$p(i,j) = \#W(i,j) \;\; \#W \tag{4}$$

$$p(i) = \#W(i) \;\; \#W \tag{5}$$

where $\#W(i)$ indicates the number of sliding windows in which word i occurs and $\#W$ denote the total number of sliding windows.

For the construction of the graph, the number of nodes is defined by $|V| = d_{size} + v_{size}$, where d_{size} is the document set, and v_{size} is the vocabulary size of the corpus (unique words). For the application of the related word filter (RWF), we assume that the input is a document $doc_i = \{sent_1, sent_2, \cdots, sent_m\}$, which consists of m sentences. A set of the word pairs $\mathcal{L} = (w_i, w_j)$ existing in the $sent^{th}$ sentence is created. For each pair of words, the related word filter (RWF) is applied based on one of the possible text representations. If there is a syntactic or semantic relationship between the word pair, then RWF = 1.

The creation of the set of edges between words occurs only when there is a value of RWF = 1. The weight assigned to the edges between the word pairs is:

$$\mathcal{E}_{i,j} \left\{ \mathcal{W}_{norm} = \mathcal{O}(w_i, w_j) \;/\; \mathcal{T}(w_i, w_j) \right. \tag{6}$$

where $\mathcal{W}_{norm}$ is the weight of the edge in the word pair w_i and w_j with a Max-Min normalization, $\mathcal{O}$ represents the number of positive RWF occurrences for this pair of words in all corpus sentences, and $\mathcal{T}(w_i, w_j)$ is the total occurrences of the word pair in the corpus. In terms of the document/word relationship, the same strategy applied in the TGCN used, considering the application of the TF-IDF to determine the existence of an edge between a word node and a document node. Formally, the set of edge weights between a node i and a node j is:

$$\mathcal{E}_{ij} = \begin{cases} \text{TF-IDF}_{ij} & i \text{ is the document, } j \text{ is the word} \\ RWF(i,j) & i,j \text{ are words, } RWF(i,j) = 1 \\ 1 & i = j \\ 0 & \text{otherwise} \end{cases} \tag{7}$$

The filter determines whether an edge exists between two word nodes based on their dependency relationship $\mathcal{R}_{dep}$. If there is a relationship, the PMI value of the terms is calculated, and if it is positive, the edge is considered. In case PMI ≤ 0, there is no relationship to be considered for the pair of words.

$$\mathcal{R}_{dep}(i,j) = \begin{cases} 1 & \text{PMI}(i,j) > 0 \\ 0 & \text{PMI}(i,j) \leq 0 \end{cases} \tag{8}$$

Figure 2 illustrates a small example of the text graph before applying the filter. The nodes with numbers represent the documents, while the others represent the word nodes. Each edge represents a direct connection between the words or a given word and the documents where it occurs. There are several edges with a weight close to 0. It is also possible to infer that the standard technique adopted in TGCN generates large amounts of edges between terms. Due to the need for the model to use the adjacency matrix, this modeling is computationally very expensive. In the computational application, if we consider each element of $\tilde{A}$ as being binary, each of them would require 1 bit of information. In a graph of size $N = 10^6$, there would be N^2 elements and thus 1 bit $\times 10^6 \times 10^6 = 10^{12}$ bits = 125 gigabytes of computational memory would be needed, since each byte corresponds to 8 bits, and Giga represents 10^9. However, such an amount of memory is prohibitive for computational applications.

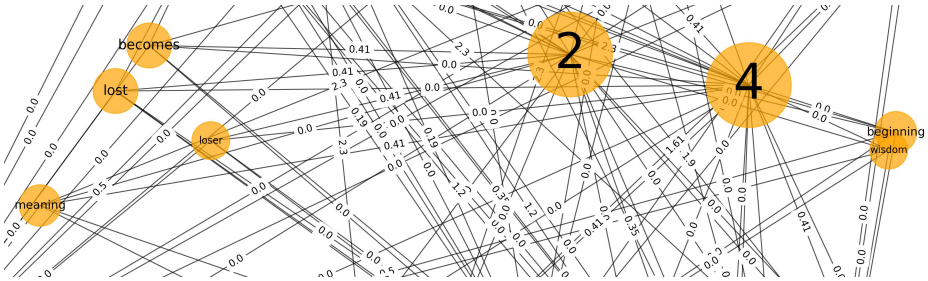


Fig. 2. Simplified text graph before applying the filter. The numbers correspond to documents, and the values on the edges are the normalized weights.

2.4 Text Representation and Filtering Modules

The application of the related word filter depends on a text representation module, which allows extracting information about the word relationships. Recently,

there have been several attempts to perform efficient graph-based text representations [19]. The connections between words have great expressive power, given that their meaning is understood based on the context. With this, there was the exploration of text representations that can consider not only co-occurrence information, but also dependency structures between the terms:

- **SYNTACTIC** (Dependency Parsing). To apply the dependency parsing filter, we used Stanford CoreNLP [22], to extract the syntactic dependency tree between words. For a given document the sentences were extracted and the dependency relation extraction applied. We built a set of word pairs where dependency occurs.
- **LINGUISTIC** (Linguistic Inquiry and Word Count - LIWC). The Linguistic Inquiry and Word Count [23] was used to extract a dependency relationship. First, we applied the parser on each sentence of a given document and, based on the result, we created a set of word pairs that match within the dictionary.
- **SEMANTIC** (Word2Vec with Skip-Gram). Word2vec was applied due to its ability to capture semantic information for word representations. All documents in the corpus were used and word2vec trained with 20 epochs and 300 dimensions with Skip-Gram. To determine whether a pair of words has a semantic dependency, the cosine similarity was used. To ensure a high dependency ratio, we only considered similarity values greater than 95.

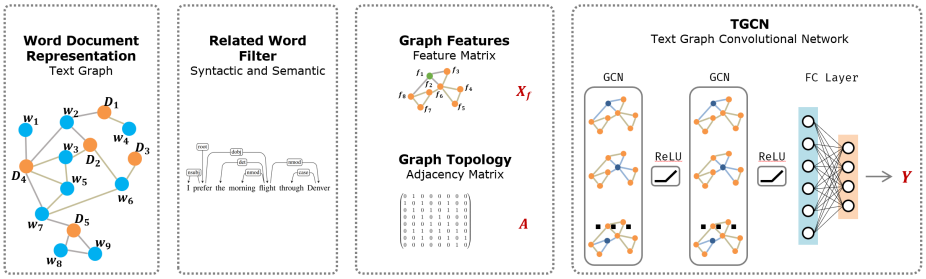


Fig. 3. The architecture of TGCN with RWF. In the generated text representation graph, the word nodes w_i have edges with each other and with document nodes D_i.

3 Performance Assessment and Results

In this work, we evaluated the standard TGCN model with a baseline, then applied the related word filter to the modules presented in Section 2.4. Specifically, we wanted to observe the behavior of the TGCN in relation to the application of the filter in its different modules of textual representation.

3.1 Datasets

The datasets used in the experiments are widely chosen in benchmark tasks for English text classification [6]. These datasets have a variety in terms of the number of documents, classes, and context to help validate the robustness of the model. Table 1 provides information about the number of documents and vocabulary, which, when added, represent the number of nodes needed to build a representation of the dataset into a single heterogeneous graph.

The values occur after a data cleaning step by means of a simple NLTK default stopword removal application [13], a set of removal of words with frequency less than 5.

Table 1. Datasets overview

Dataset	Docs	Words	Nodes	Train	Test	Classes
20NG	18,846	42,757	61,603	11,314	7,532	20
MR	10,662	18,764	29,426	7,108	3,554	2
R8	7,674	7,688	15,362	5,485	2,189	8
R52	9,100	8,892	17,992	6,532	2,568	52
CORA	2,708	1,343	4,051	1,896	812	7
PUBMED	19,717	452	20,169	13,802	5,915	3
CITESEER	3,312	3,515	6,827	2,319	993	6

3.2 Implementation

In this experiment, we also use two GCN layers, each one with a dropout rate of 0.5 and using 200 for dimension of the node embedding in the first layer. The first layer applies a ReLU activation to the output from a 200-dimension hidden layer, while the final layer applies the Softmax operation on the output layer. The number of outputs is defined by the number of classes. The Adam Optimiser is used with a learning rate of 0.02 and the models are trained for a maximum of 200 epochs. Early stopping is performed when validation loss does not decrease for 10 consecutive epochs. We randomly selected 10% of the training set as the validation set, whose labels are not used for training, for all datasets above. The implementation uses linear layers developed in PyTorch [14]. The architecture of TGN with the RWF is shown in Figure 3, where FC is the Fully Connected layer. The code to reproduce all experiments, with datasets can be found online [24]. The LIWC lexicon is proprietary, so it is not included in this repository.

Baselines To better compare the performance of the filters applied, we used the following methods as a baseline:

– TGCN: We used the implementation and parameters adopted in the standard TGCN, as presented by [10].

- CNN: Convolutional Neural Network [5]. We explored CNN to perform convolution and max pooling operation on word and max pooling embeddings to get representation of text.
- LSTM: The LSTM model defined in [12] which uses the last hidden state as the representation of the whole text. We also experimented with the model with/without pre-trained word embeddings.

3.3 Results and Analysis

Table 2 presents the values in relation to the number of edges generated in the construction of the text graph representation. In the pre-processing stage, we store the adjacency matrices in files that will be used in the training stage, creating the array that will be consumed as an input to the TGCN network. To represent all the resources present in the text graph, it is necessary to create an array with the data type Double-precision floating-point format (float64). For comparison purposes, the 20NG dataset adjacency matrix generates a 296MB file. For its computational representation, it will be necessary to allocate 28.2GB of memory.

In our experiments, applying the filter on the connection of word nodes and document nodes showed a significant decrease in terms of performance. Both the application of the TW-IDF and the creation of a threshold on the result of the TF-IDF, reduce the representativeness of terms, and therefore, in some aspects, make the syntactic and semantic extraction proposed by the text filter unfeasible. An attempt to apply the TW-IDF method [18] showed a decrease of $\approx$70% in terms of test accuracy in the worst case.

Regarding performance, the semantic filter obtained the best results, with the exception of the CORA dataset. In general, the syntactic filters obtained similar results, indicating that a general approach can be applied, varying between different representations as in the case of Constituency Parsing.

Table 3 presents the test accuracy results for the standard TGCN used as a baseline, and for the filter-based models. As can be seen, the semantic filter model obtained better results for the R8 and Pubmed datasets. The LIWC based syntactic filter model and Dependency Parsing vary with the second-best performance for 4 datasets. In both cases, the results achieved by the filter models exceed the CNN and LSTM baseline. In general, the application of the filter was able to achieve results close to or greater than the TGCN in the best cases, with a high reduction in terms of memory and processing time.

Although TGCN-filtered does not significantly outperform the standard TGCN in all cases, it has reduced memory consumption and the training time, which makes this model more computationally attractive.

4 Conclusions

This paper introduced the generation of a new text graph for classification activities based on neural networks for graphs. As the main contribution, we substantially reduced the size of the text graph without compromising performance

Table 2. Comparison of memory consuming. Number of edges of the text graph.

MODEL	20NG	CITESEER	CORA	MR	PUBMED	R52	R8
TGCN	24,689,966	1,106,622	335,138	1,701,424	929,469	3,981,246	3,165,430
SEMANTIC	2,286,228	173,530	73,336	226,468	849,459	407,694	324,464
SYNTACTIC	3,130,672	105,276	49,344	407,508	850,473	682,675	548,166
LINGUISTIC	2,529,518	104,444	48,918	219,116	850,107	492,946	393,562

Table 3. Accuracy on text classification datasets.

MODEL	20NG	CITESEER	CORA	MR	PUBMED	R52	R8
TGCN	0.86126	0.73313	0.75616	0.76027	0.85866	0.93497	0.96848
CNN	0.76931	0.68236	0.69584	0.74981	0.79581	0.85377	0.94037
LSTM	0.65712	0.69281	0.71064	0.75064	0.82984	0.85543	0.93682
SEMANTIC	0.85068	0.71098	0.71921	0.72651	0.86052	0.92445	0.96895
SYNTACTIC	0.84665	0.71702	0.71552	0.73438	0.85123	0.91978	0.95021
LINGUISTIC	0.84241	0.69889	0.72906	0.72313	0.85968	0.91783	0.95432

and in the best of cases, with a performance superior to that presented by the TGCN. The high performance of GCNs in NLP tasks has outperformed numerous state-of-the-art methods on multiple benchmark datasets. Its robustness, combined with a correct selection of data, allows high performance with reduced consumption in terms of memory and training time.

Some interesting future directions include improving the generation of text graphs that are more representative in terms of context, as well as alternatives for processing subgraphs, which would make it feasible to work with large volumes of data.

References

1. Minaee, Shervin, Nal Kalchbrenner, Erik Cambria, Narjes Nikzad, Meysam Chenaghlu, and Jianfeng Gao. "Deep Learning–based Text Classification: A Comprehensive Review." ACM Computing Surveys (CSUR) 54, no. 3 (2021).
2. Aggarwal, Charu C., and ChengXiang Zhai. "A survey of text classification algorithms." In Mining text data, pp. 163-222. Springer, Boston, MA, (2012).
3. Pennington, Jeffrey, Richard Socher, and Christopher D. Manning. "Glove: Global vectors for word representation." In Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP), pp. 1532-1543. (2014).
4. Schuster, Mike, and Kuldip K. Paliwal. "Bidirectional recurrent neural networks." IEEE transactions on Signal Processing 45, no. 11 (1997).
5. Yoon Kim. 2014. "Convolutional neural networks for sentence classification." In Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), pages 1746–1751 (2014).
6. Kipf, Thomas N., and Max Welling. "Semi-supervised classification with graph convolutional networks." arXiv preprint arXiv:1609.02907 (2016).
7. Yang, JinXiong, Liang Bai, and Yanming Guo. "A survey of text classification models." In Proceedings of the 2020 2nd International Conference on Robotics, Intelligent Control and Artificial Intelligence, pp. 327-334. (2020).

8. François Rousseau, Emmanouil Kiagias, and Michalis Vazirgiannis. "Text categorization as a graph classification problem." In ACL, Vol. 15. 107 (2015).
9. Veličković, Petar, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. "Graph attention networks." arXiv preprint arXiv:1710.10903 (2017).
10. Yao, Liang, Chengsheng Mao, and Yuan Luo. "Graph convolutional networks for text classification." In Proceedings of the AAAI conference on artificial intelligence, vol. 33, no. 01, pp. 7370-7377. (2019).
11. Luo, Yuan, Özlem Uzuner, and Peter Szolovits. "Bridging semantics and syntax with graph algorithms—state-of-the-art of extracting biomedical relations." *Briefings in bioinformatics* 18, no. 1 (2017).
12. Liu, Pengfei, Xipeng Qiu, and Xuanjing Huang. "Recurrent neural network for text classification with multi-task learning." In Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence, pp. 2873-2879. (2016).
13. Bird, Steven. "Multidisciplinary instruction with the natural language toolkit." Association for Computational Linguistics, (2008).
14. Paszke, Adam, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. "Automatic differentiation in pytorch." (2017).
15. Wu, Lingfei, Yu Chen, Kai Shen, Xiaojie Guo, Hanning Gao, Shucheng Li, Jian Pei, and Bo Long. "Graph neural networks for natural language processing: A survey." *arXiv preprint arXiv:2106.06090* (2021).
16. Role, Francois, and Mohamed Nadif. "Handling the impact of low frequency events on co-occurrence based measures of word similarity." In *Proceedings of the international conference on Knowledge Discovery and Information Retrieval (KDIR-2011). Scitepress*, pp. 218-223. (2011).
17. Roelleke, Thomas, and Jun Wang. "Tf-idf uncovered: a study of theories and probabilities." In *Proceedings of the 31st annual international ACM SIGIR conference on Research and development in information retrieval* , pp. 435-442. (2008).
18. Rousseau, François, and Michalis Vazirgiannis. "Graph-of-word and TW-IDF: new approach to ad hoc IR." In *Proceedings of the 22nd ACM international conference on Information & Knowledge Management* , pp. 59-68. (2013).
19. Osman, Ahmed Hamza, and Omar Mohammed Barukub. "Graph-based text representation and matching: A review of the state of the art and future challenges." IEEE Access 8 (2020).
20. Jin, Wei, and Rohini K. Srihari. "Graph-based text representation and knowledge discovery." In Proceedings of the 2007 ACM symposium on Applied computing, pp. 807-811. (2007).
21. Wang, Yujing, Xiaochuan Ni, Jian-Tao Sun, Yunhai Tong, and Zheng Chen. "Representing document as dependency graph for document clustering." In Proceedings of the 20th ACM international conference on Information and knowledge management, pp. 2177-2180. (2011).
22. Manning, Christopher D., Mihai Surdeanu, John Bauer, Jenny Rose Finkel, Steven Bethard, and David McClosky. "The Stanford CoreNLP natural language processing toolkit." In Proceedings of 52nd annual meeting of the association for computational linguistics: system demonstrations, pp. 55-60. (2014).
23. Pennebaker, James W., Martha E. Francis, and Roger J. Booth. "Linguistic inquiry and word count: LIWC 2001." Mahway: Lawrence Erlbaum Associates 71 (2001).
24. Github https://github.com/vitormeriat/nlp-based-text-gcn. Last accessed 6 June 2022